TEKEVER Data Intelligence Award — Technical Report

Formula Manipal · Class 1 EV · Formula Student Portugal · Submitted 05/07/2026

All telemetry in this report is real data logged on our car. The single exception is the steering angle channel, which is declared synthetic and described in section 8.

1. System Architecture

Our telemetry system is organised in three stages.

Acquisition. A Teensy 4.1 telemetry ECU logs 67 parameters per row at 10 Hz to a 64 GB SD card, roughly 6 MB per endurance run. Data arrives from the 500 kbps vehicle CAN bus, which carries the PM100DZ motor controller and our AMS Master battery management system, and from a VectorNav VN-200 IMU/GNSS unit read over USB at 50 Hz. Each subsystem keeps its own clock, so the raw file contains seven independent timestamp columns.

Preprocessing. A ten-step Python pipeline (preprocess.py) takes each raw file, aligns all clocks to the master, resamples everything onto a common 10 Hz grid, cleans out bad samples, converts to SI units and adds derived engineering channels. The result is one calibrated file per run.

Analysis and access. A batch engine (analytics.py) computes our KPI set over every run and writes the results out as CSVs and plots. Day to day we work through three routes: the run catalogue filter described in section 2, the exported Level 1 and Level 2 files, and a Jupyter notebook for interactive digging.

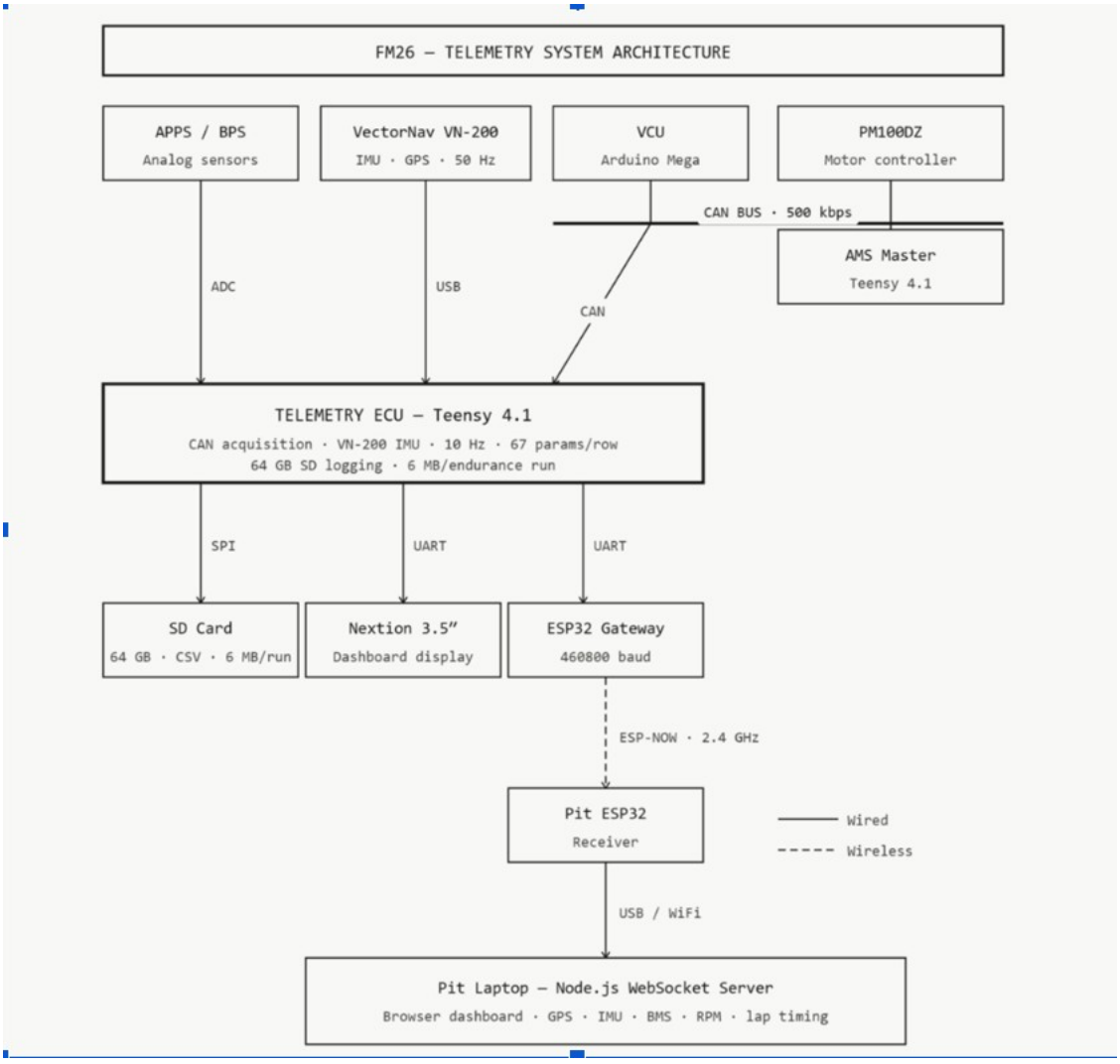

Figure 1 — FM26 telemetry system architecture.

2. Data Model: Organisation, Indexing and Search

Every run is a row in a run catalogue, so a run can be found and checked without opening its data. The catalogue keeps two kinds of metadata.

The first kind is computed straight from the data by the catalogue builder: duration, distance (with its source), maximum speed, peak acceleration, which channels are present, and a validity flag. Since nobody types these in, they cannot drift out of sync with the data.

The second kind is what only the team knows, kept in one editable table and merged in by run: date, event, driver, session type, car configuration (aeropack, springs, dampers, anti-roll bars) and whether the data is real or synthetic.

Field	Source	Example
run_id	derived	2025-05-12_Endurance_D2_01
date, event	human	2025-05-12, Endurance
driver	human	D2
session_type	human (a guess is auto-suggested)	Endurance / Skidpad / AutoX / Practice
car_config	human	aeropack=Y; springs; dampers; ARB
duration_s, distance_km, max_speed_kmh, peak_accel_g	derived	—
valid	derived	true / false
origin	human	real / synthetic
file_path	derived	link to raw and Level-1 data

Search. The catalogue can be filtered on any combination of fields, for example all Endurance runs with the aeropack fitted and driver D2. Each result carries its file path, so a judge can pick any hit, open the underlying file and check that the metadata matches the data.

Traceability. Every record links one-to-one to its raw and calibrated files. Setup changes live in the car_config fields, so a change in performance can be traced back to the configuration that produced it.

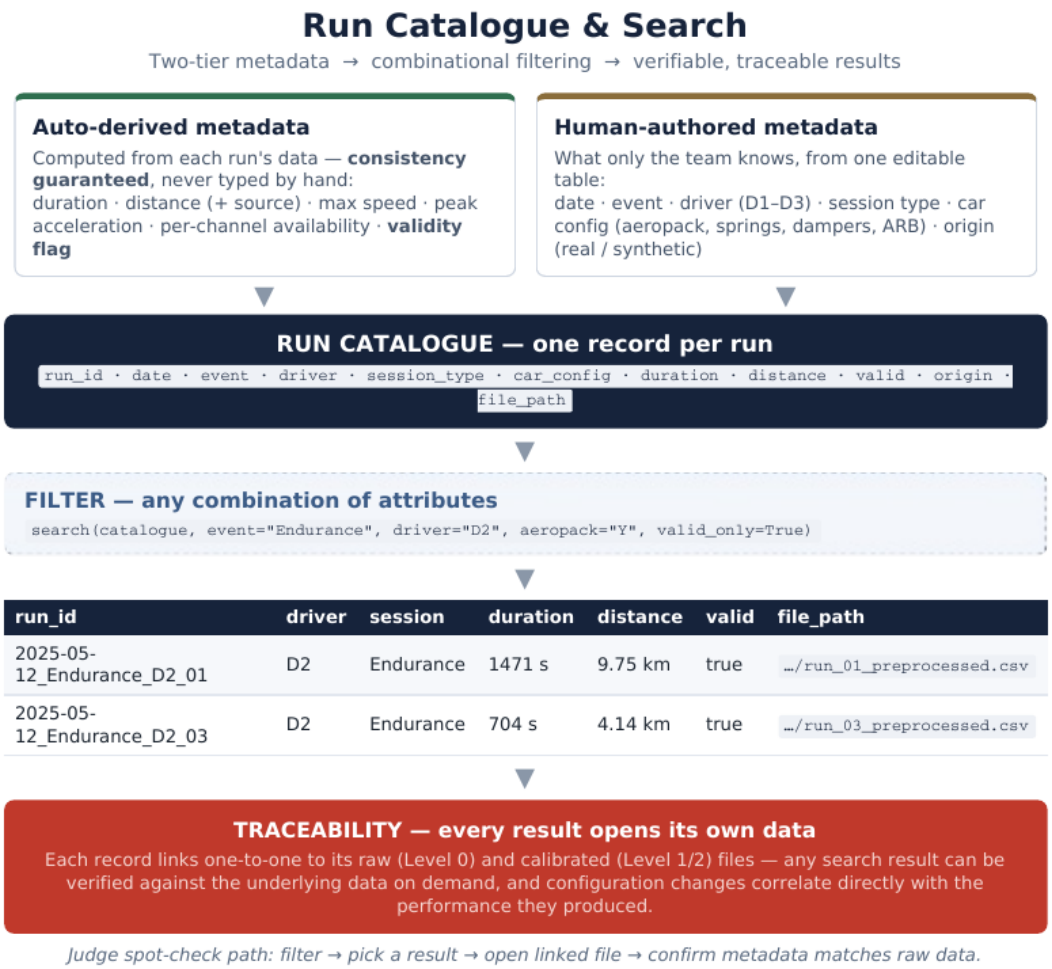

Figure 2 — catalogue structure and the search-to-verification path.

3. Pre-computed KPIs

The four mandatory EV KPIs are computed for every valid run. Each one checks its input channels first; if something is missing, the KPI reports which channel and why instead of returning a silent zero.

KPI	Unit	Definition	Fault reported
Total energy	Wh	Σ(V_dc · I_dc · Δt) / 3600	DC bus voltage or current channel absent
Energy per km	Wh/km	total energy / distance	distance under 100 m, or energy unavailable
Distance	km	Σ(v · Δt); v is GNSS speed, or motor-derived ground speed when GNSS is missing	no speed source present
Run duration	s	last value of the synchronised time base	time base missing

On top of these we pre-compute maximum inverter temperature, energy recovered through regen, maximum speed, initial and final state of charge from pack voltage, and peak combined acceleration (the 99th percentile of the combined longitudinal and lateral acceleration, reported in m/s² and g). A wider derived set covers power and efficiency, friction-circle grip use, thermal rise rates, torque-response lag, drivetrain slip and lap segmentation.

Our valid-run count is well past the ten runs needed for full coverage.

Two notes on method. Peaks are computed on the 10 Hz grid, so they slightly understate true instantaneous peaks. Distance uses GNSS speed where we have it and falls back to a motor-derived estimate where we do not;

on runs carrying both, the fallback agrees with GNSS to within about 6%, and every run records which source was used.

4. Defining and Computing New KPIs Retroactively

A KPI in our system is just a Python function over the standard per-run dataframe. Each one returns a value, a status and a list of any missing channels, and each one checks channel availability before it runs. Because every run shares the same Level-1 schema, adding a new KPI means writing one function; the batch engine then applies it across every historical run and re-exports the results. This is exactly what the live challenge asks for: a judge defines a KPI, we write one function, and it computes retroactively over as many valid runs as needed, without touching storage or re-ingesting anything.

5. Data Access and Export

Level	Definition	In our system
0 — Raw	Per-sensor native files, unsynchronised, counts/volts	Raw SD-card files with per-module timestamps
1 — Calibrated	Single unified file, clocks synchronised, SI units	Drift-corrected sync, uniform 10 Hz grid, SI units
2 — Derived	Channels computed with knowledge of the car	Power, efficiency, energy, slip, grip / friction circle, lap segmentation

Any time interval of a run can be exported at Level 2 — for example the last 30 seconds of an Endurance run with all derived channels included — by slicing on the run time base.

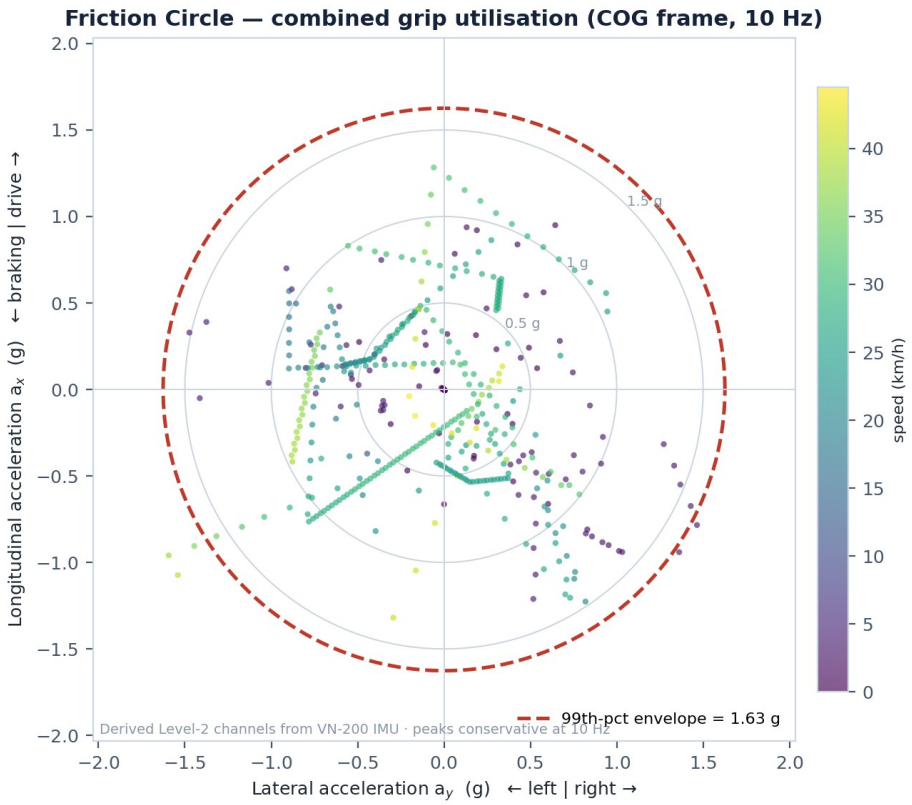

Figure 3 — friction circle for our highest-grip run. The dashed ring is the 99th-percentile combined acceleration (1.63 g), the same statistic used by the peak-acceleration KPI.

6. Data Quality, Error Logging and Integrity

Quality checks run at every stage of the pipeline:

- **Ingestion checks** — expected columns, timestamp de-duplication and monotonicity, a minimum-duration guard that rejects corrupt or truncated files, and per-column completeness recorded for every run.
- **Range and sentinel checks** — physical limits per channel (motor speed 0–6500 rpm, phase current ±200 A, vehicle speed 0–45 m/s); out-of-range samples and sentinel values such as −1000 are nulled.
- **Sign validation** — the motor controller's rotation convention can invert the apparent sign of current and power, so each run is checked against physics: while the motor is clearly spinning, median electrical power must be positive. Inverted runs are corrected automatically and flagged in a dedicated column.
- **Gap handling** — gaps under 100 ms are interpolated, gaps up to 2 s cautiously, and anything longer splits the run into segments; we never fabricate data across a long gap.
- **Stuck-sensor detection** — a channel showing zero variance over 50 or more consecutive samples is flagged as frozen and nulled.
- **Duplicate detection** — the same session ingested twice is caught, so duplicates cannot inflate the run set.
- **Hardware fault flags** — the BMS broadcasts cell over/under-voltage, temperature, over-current, communication-error and watchdog flags, all captured in telemetry, so a fault traces back to its source channel.
- **Error reporting** — failures and missing channels are logged per run and surface through the KPI fault plan in section 3; nothing fails silently.

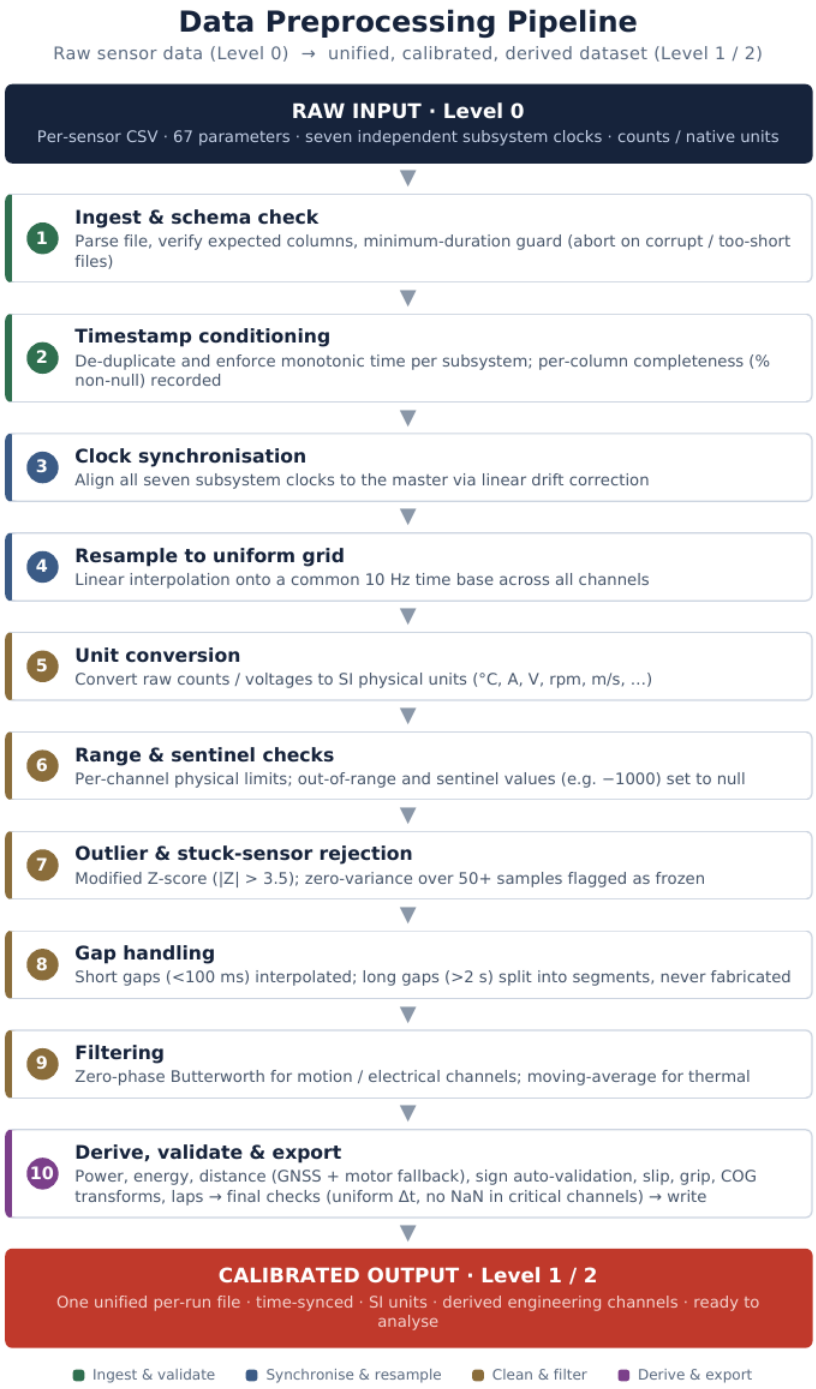

Figure 4 — the ten-step preprocessing pipeline, raw Level 0 in, calibrated Level 1/2 out.

7. Design Decisions and Trade-offs

- **One master clock.** Linear drift correction onto a single time base costs preprocessing effort, but without it we could not relate battery power draw to what the car was doing dynamically.
- **Validated sign convention instead of a hard-coded fix.** We originally assumed the DC current sensor was wired in reverse and negated it at ingestion. The per-run physics check described in section 6 replaced that assumption, because the inversion turned out to depend on the run.
- **Uniform 10 Hz grid.** Resampling everything to 10 Hz is robust for multi-rate, sparse CAN data and gives one coherent grid, at the cost of high-frequency IMU detail. As a consequence, instantaneous peaks are reported conservatively.
- **Filters matched to the signal.** Zero-phase Butterworth for motion and electrical channels, moving average for slow thermal channels.

- **Plain Python.** Everything is pandas scripts and a notebook. A command-line and notebook front end is enough for our workflow, keeps the whole system reproducible, and means nothing depends on one dashboard staying alive.

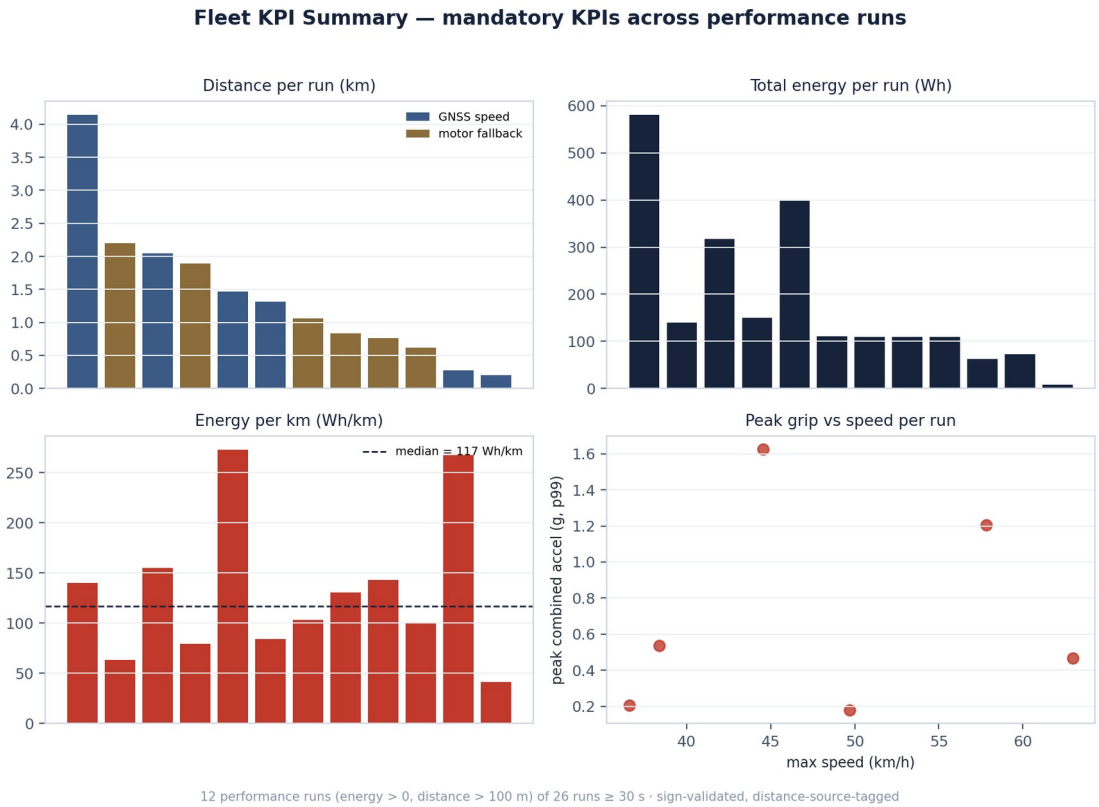

Figure 5 — mandatory KPIs across the performance runs in the catalogue. Bar colours in the distance panel show which speed source was used.

8. Complete Physical Sensor List

All channels below are real logged telemetry. The one exception is steering angle, covered in the note after the table.

#	Channel(s)	Unit	Source sensor	Interface	Rate
1	Module_A/B/C_Temp, Gate_Driver_Temp	°C	PM100DZ inverter, IGBT modules / gate driver	CAN	500 kbps, logged 10 Hz
2	Control_Board_Temp, RTD1/RTD2_Temp, Stall_Burst_Temp	°C	PM100DZ controller board / motor-winding RTDs	CAN	500 kbps, logged 10 Hz
3	Motor_Angle, Delta_Resolver	deg	Resolver via PM100DZ	CAN	500 kbps, logged 10 Hz
4	Motor_Speed	rpm	PM100DZ motor controller	CAN	500 kbps, logged 10 Hz
5	Elec_Frequency	Hz	PM100DZ motor controller	CAN	500 kbps, logged 10 Hz
6	Phase_A/B/C_Current, DC_Bus_Current	A	Phase and DC-bus current sensors via PM100DZ	CAN	500 kbps, logged 10 Hz
7	DC_Bus_Voltage, Output_Voltage, VAB_Vd, VBC_Vq	V	DC bus / controller output via PM100DZ	CAN	500 kbps, logged 10 Hz
8	Commanded_Torque, Torque_Feedback	Nm	PM100DZ, demand and actual	CAN	500 kbps, logged 10 Hz

9	Power_On_Timer	s	PM100DZ motor controller	CAN	500 kbps, logged 10 Hz
10	Yaw, Pitch, Roll	deg	VectorNav VN-200, attitude	USB	50 Hz, logged 10 Hz
11	Accel_X/Y/Z	m/s^2	VectorNav VN-200 IMU, body frame	USB	50 Hz, logged 10 Hz
12	Gyro_X/Y/Z	deg/s	VectorNav VN-200 IMU	USB	50 Hz, logged 10 Hz
13	Pos_Ex/Ey/Ez	m	VectorNav VN-200 GNSS, ECEF	USB	50 Hz, logged 10 Hz
14	Vel_Ex/Ey/Ez	m/s	VectorNav VN-200 GNSS	USB	50 Hz, logged 10 Hz
15	BMS_Seg0–4, BMS_TotalV, BMS_LowestCellV (+90 cell voltages)	V	AMS Master, 5× LTC6813, 90 cells	CAN	20 Hz measured, logged ~10 Hz
16	BMS_Current	A	AMS Master, Hall-effect pack current sensor	CAN	10 Hz
17	BMS_PackAvgTemp, BMS_LowestIC/Index, BMS_Latch, BMS_FaultFlags	°C / flag	AMS Master, pack thermistors (Orion chain)	CAN	~10 Hz
18	steering_angle	deg	Magnetic rotary angle sensor (Magneto Click)	I2C / SPI	logged 10 Hz

On steering angle. The magnetic angle sensor is newly fitted to the car and was not present when the historical sessions were logged. For those runs the steering channel is therefore synthetic: reconstructed from measured IMU lateral acceleration and speed through a single-track model, checked against the measured lateral acceleration, and marked origin = synthetic in the run catalogue. The centre-of-gravity acceleration and velocity channels (ax_cog, ay_cog, v_mag_cog and so on) are transformed from the VN-200 and count as derived channels, not separate sensors, which is why they do not appear in the table above.

9. Glossary

- **Run** — one continuous logged session, i.e. one SD-card file, mapped one-to-one to its calibrated file. Outlaps are not separated from timed laps at the run level; laps within a run are identified by the lap-segmentation channel. Test sessions and competition sessions are distinguished by the session_type field.
- **Valid run** — a run containing the channels needed for all mandatory KPIs, plus steering angle and at least three axes of IMU acceleration.
- **Naming** — runs are named YYYY-MM-DD_EVENT_DRIVER_NN, for example 2025-05-12_Endurance_D2_01.
- **Drivers** — D1 Ishaan, D2 Shadanan, D3 Rishi.
- **KPI definitions** — the mandatory formulas are in section 3; the full derived-set formulas travel with the analytics code and figure captions.